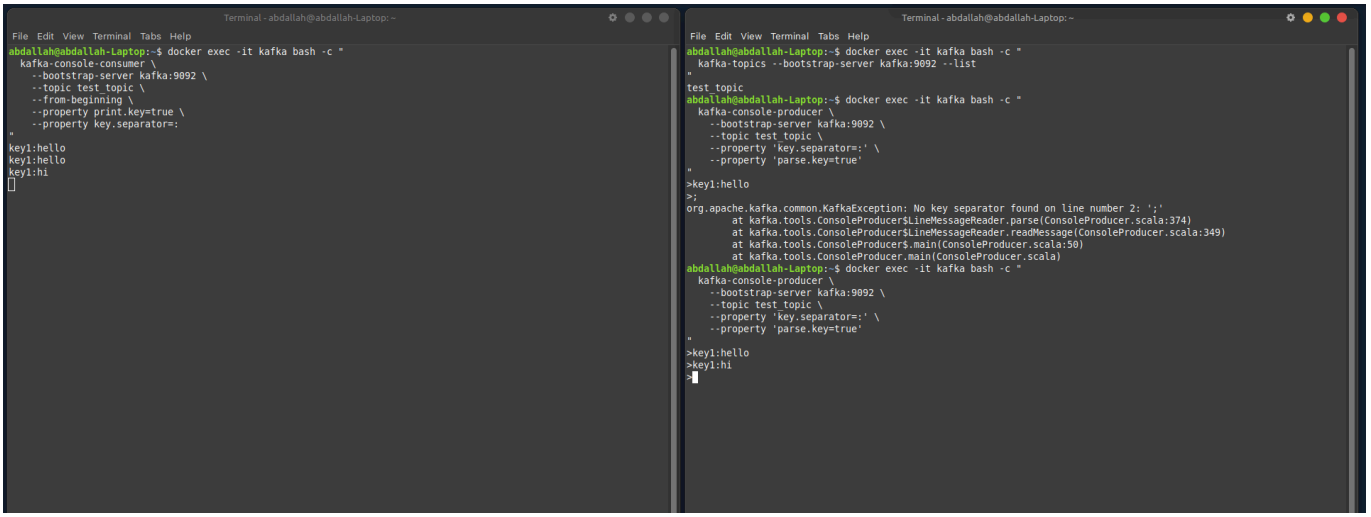
Q1:

Summary: Create a new Kafka topic named `test_topic` and launch a Kafka console producer configured to parse input as key-value pairs separated by a colon (`:`). **Commands Executed:**

```
# Topic creation step for Q1
docker exec -it kafka kafka-topics --create --topic test_topic --bootstrap-server kafka:9092

# Commands visible in the screenshot
docker exec -it kafka bash -c "
  kafka-topics --bootstrap-server kafka:9092 --list
"

docker exec -it kafka bash -c "
  kafka-console-producer \
    --bootstrap-server kafka:9092 \
    --topic test_topic \
    --property 'key.separator=:' \
    --property 'parse.key=true'
"
```

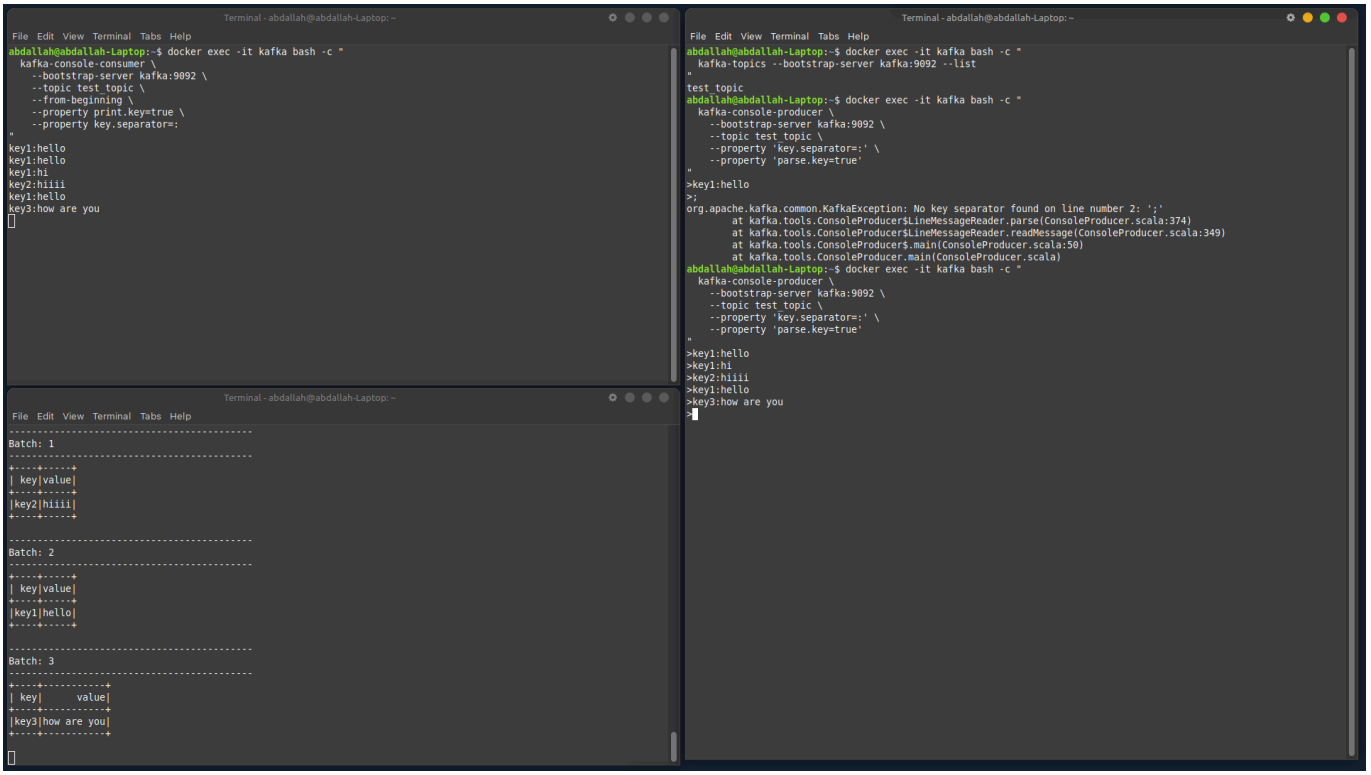


Q2:

Summary: Launch a Kafka console consumer to read messages from `test_topic` from the beginning, ensuring the console is formatted to display both the key and the value separated by a colon. **Command Executed:**

```
docker exec -it kafka bash -c "
  kafka-console-consumer \
    --bootstrap-server kafka:9092 \
    --topic test_topic \
    --from-beginning \
    --property print.key=true \
```

```
    --property key.separator=:
"
```

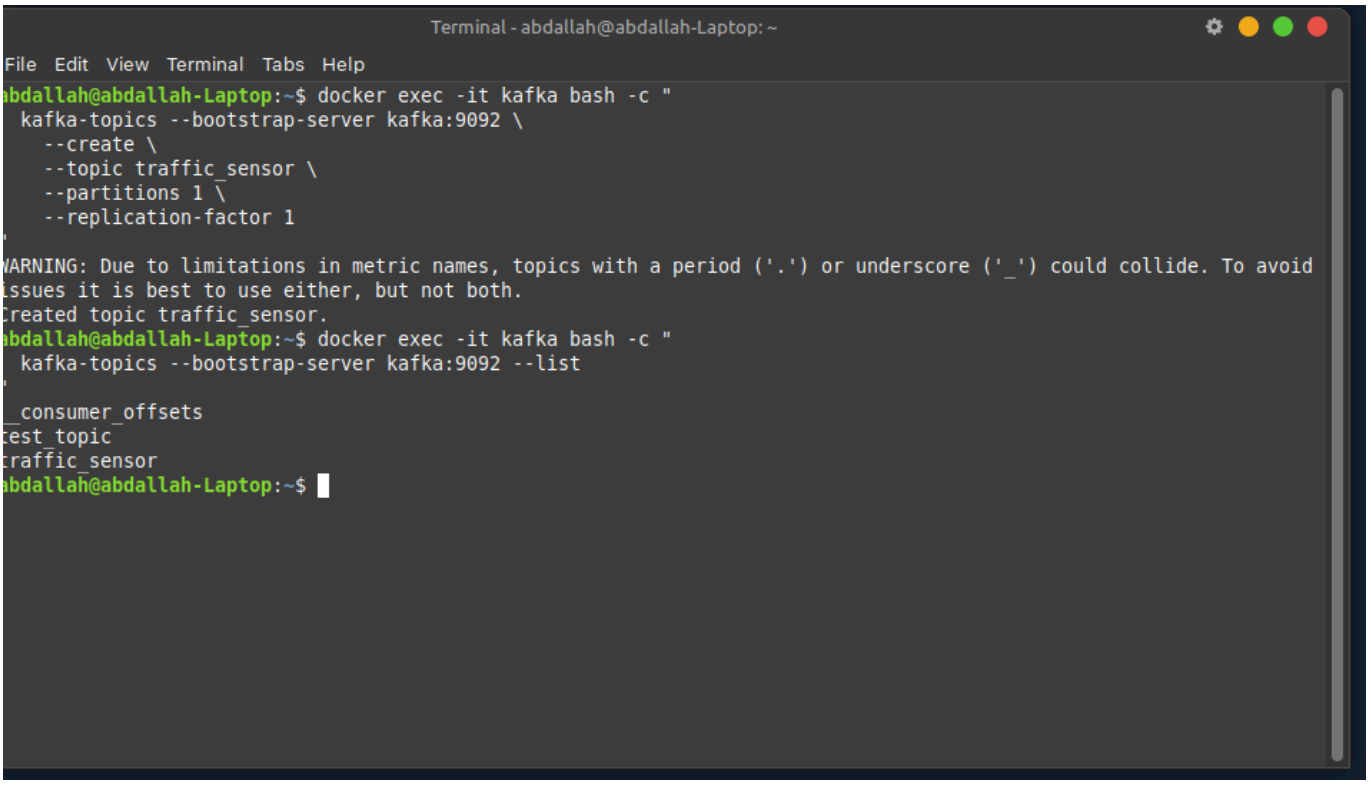


Q3:

Summary: Create the main Kafka topic named `traffic_sensor` which will be used to ingest and stream the dataset's traffic records for the remainder of the lab. **Command Executed:**

```
docker exec -it kafka bash -c "
  kafka-topics --bootstrap-server kafka:9092 \
    --create \
    --topic traffic_sensor \
    --partitions 1 \
    --replication-factor 1
"

docker exec -it kafka bash -c "
  kafka-topics --bootstrap-server kafka:9092 --list
"
```



Q4/5/6:

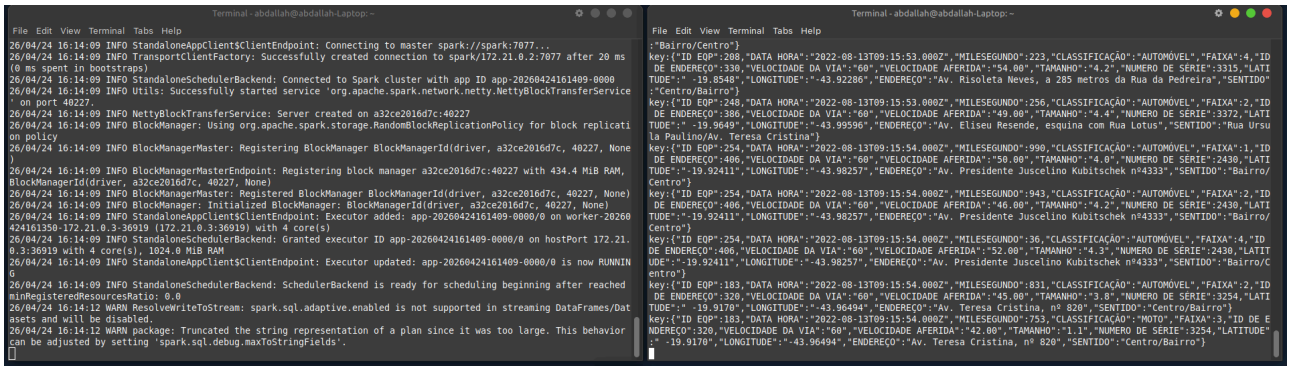
Summary:

- **Q4:** Start a consumer listening to the `traffic_sensor` topic from the beginning.
- **Q5:** Run a Python producer script (`write_traffic_streamQ5.py`) via Spark to quickly send a bulk batch of 500,000 historical traffic JSON records to the Kafka topic.
- **Q6:** Observe the consumer terminal successfully receiving the flood of messages. **Commands Executed:**

```
# Q4 Consumer (Right Terminal)
docker exec -it kafka kafka-console-consumer --topic traffic_sensor --
bootstrap-server kafka:9092 --from-beginning

# Q5 Producer (Left Terminal)
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master
spark://spark:7077 --packages org.apache.spark:spark-sql-kafka-0-
10_2.12:3.3.2 /src/streaming/write_traffic_streamQ5.py"
```

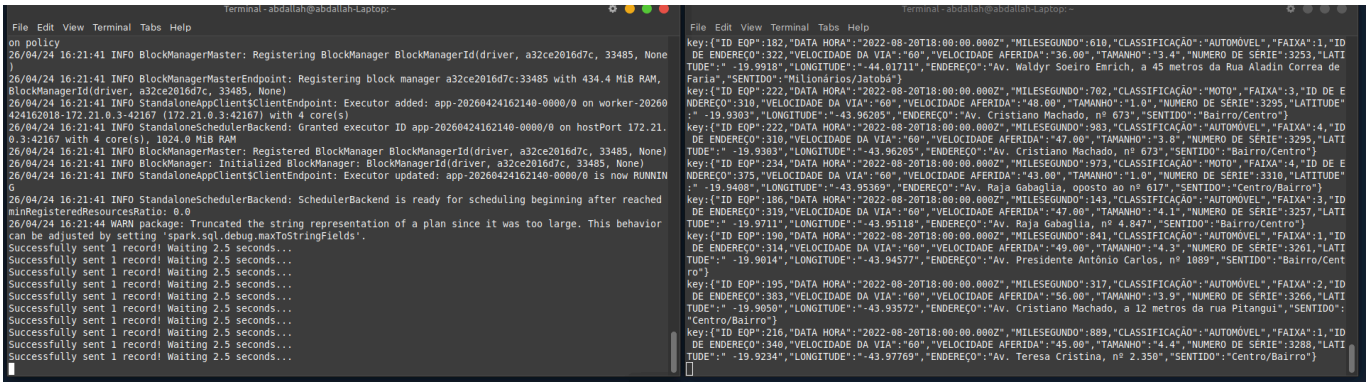
- the terminal on the right is for Q4 and the terminal on the left is the execution of the script `write_traffic_streamQ5.py` for Q5

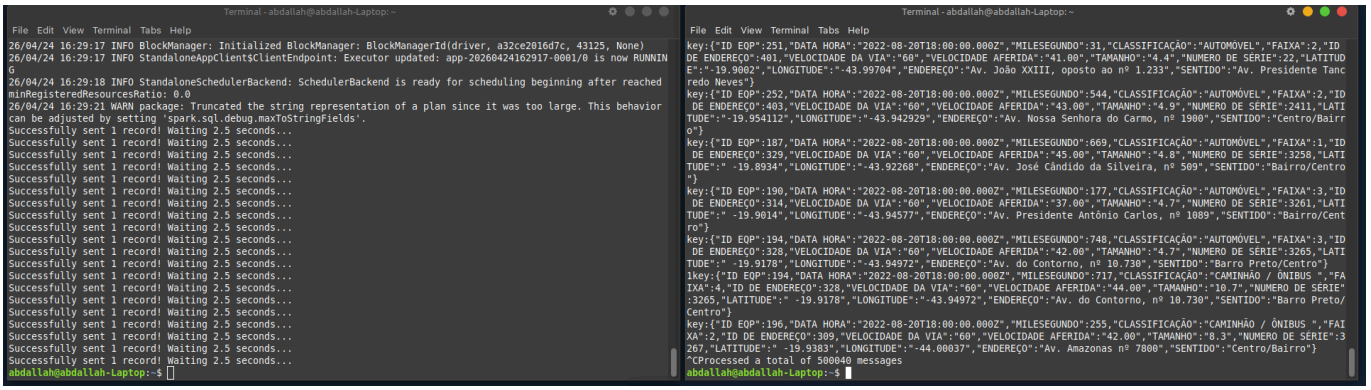


Q7:

Summary: Simulate a real-time hardware sensor by executing a modified PySpark script (`write_traffic_batch.py`). Instead of sending records instantly, this script reads the data and sends 1 record every 2.5 seconds to the `traffic_sensor` topic. **Python File:** `write_traffic_batch.py`
Command Executed:

```
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master spark://spark:7077 --total-executor-cores 2 --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2 /src/streaming/write_traffic_batch.py"
```



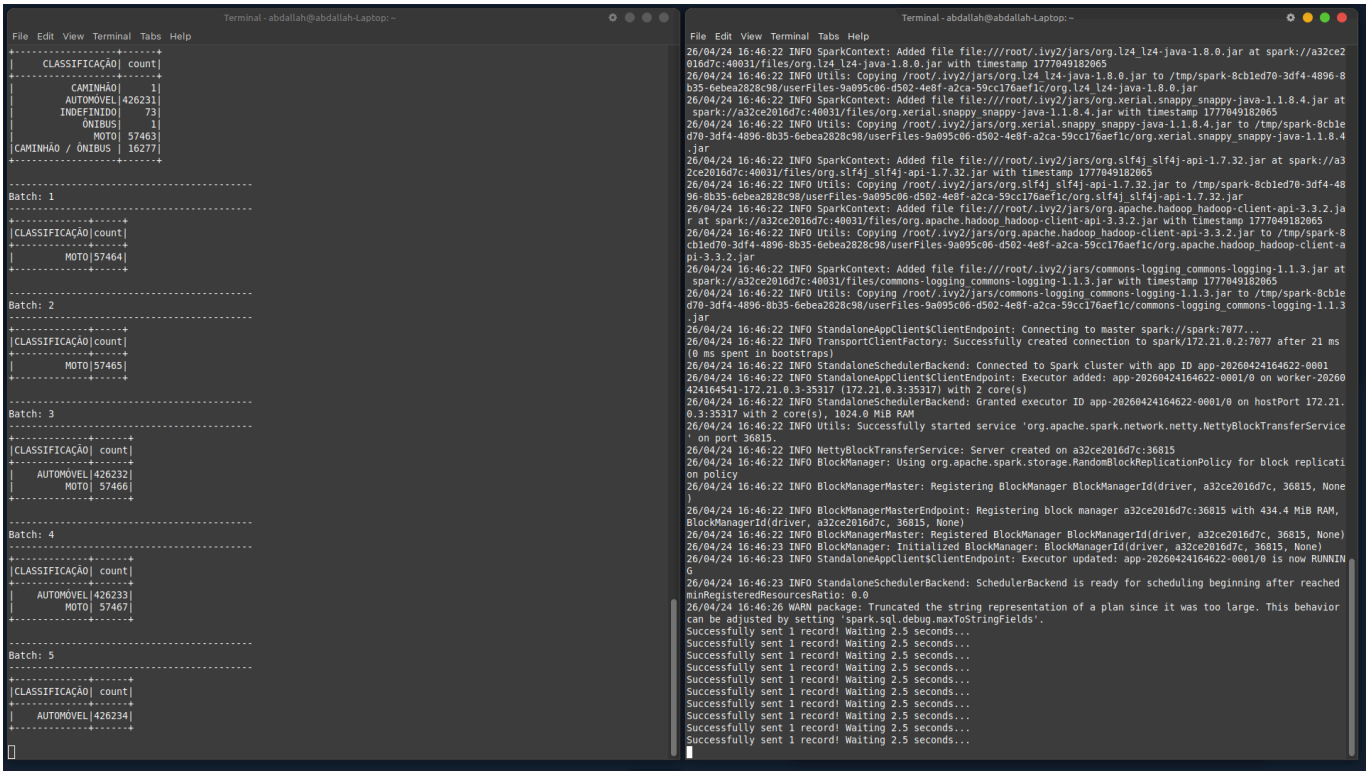


- as shown in the terminal, the script keeps sending one record every 2.5 seconds until it reaches the limit of 20 records
- the Kafka topic still contains the data produced in Q5; because Q4 uses `--from-beginning`, the consumer first reads the existing 500,000 records from Q5 and then receives the 20 new records produced by the Q7 script

Q8:

Summary: Create a PySpark Structured Streaming application to act as a consumer. It continuously reads the live data stream from Kafka, parses the JSON, and dynamically counts the total number of vehicles grouped by their classification type (e.g., AUTOMÓVEL, MOTO) in Update Mode. **Python File:** `count_vehicles.py`
Command Executed:

```
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master spark://spark:7077 --total-executor-cores 2 --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2 /src/streaming/count_vehicles.py"
```



- as we see in the right terminal for Q7 keep acting as sencor and sent messages every 2.5 sec and the Q8(the left terminal) will update the couter when he recive data

Q9:

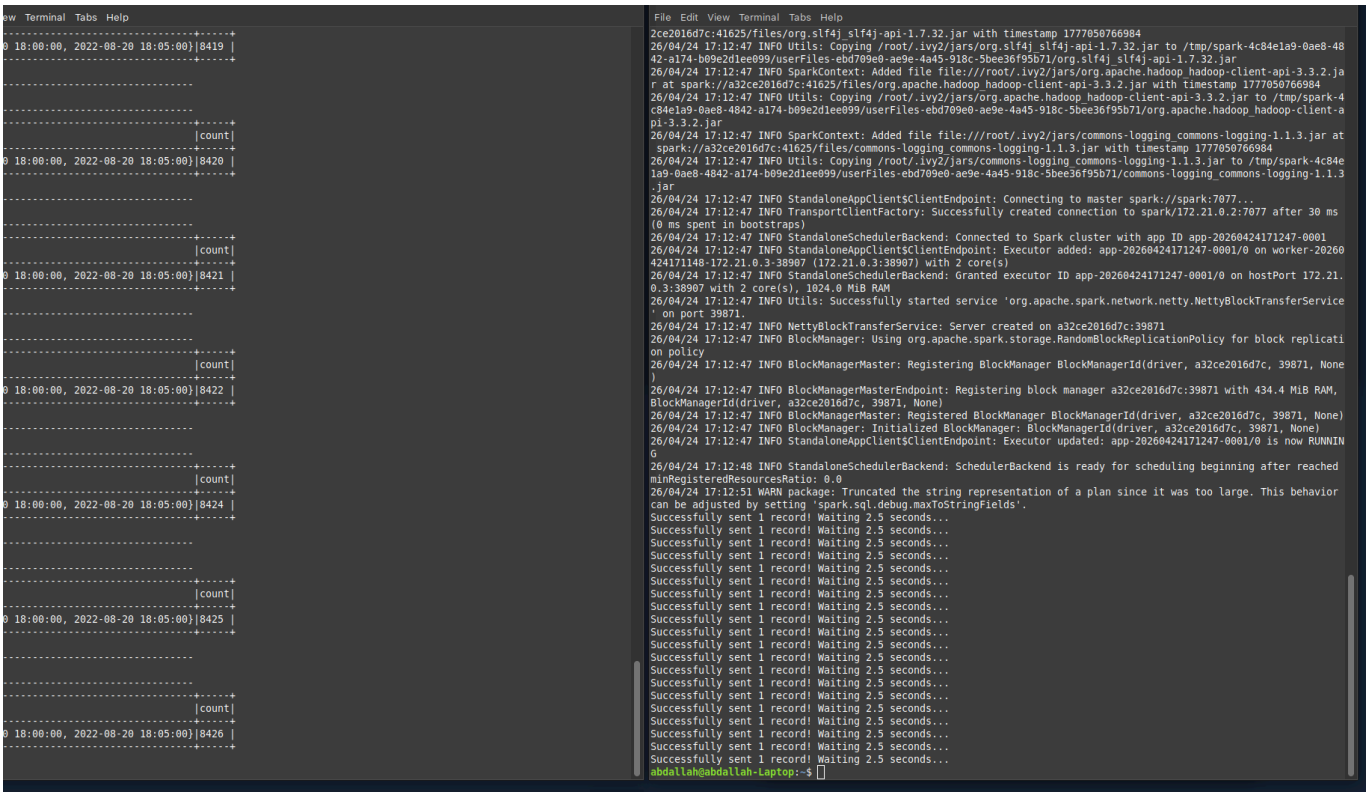
Summary: Write a streaming query to process the data using an Event-Time Window. The application groups the incoming traffic data into fixed, non-overlapping **1-minute Tumbling Windows** based on the `DATA HORA` timestamp, and calculates the total count of vehicles per window. **Python File:** `tumbling_count.py`
Command Executed:

```
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master spark://spark:7077 --total-executor-cores 2 --packages org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2 /src/streaming/tumbling_count.py"
```

Terminal - abdallah@abdallah-Laptop: ~

File Edit View Terminal Tabs Help

```
26/04/24 17:24:48 INFO StandaloneAppClient$ClientEndpoint: Connecting to master spark://spark:7077...
26/04/24 17:24:48 INFO TransportClientFactory: Successfully created connection to spark/172.21.0.2:7077 after 26 m
(0 ms spent in bootstraps)
26/04/24 17:24:48 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20260424172448-0005
26/04/24 17:24:48 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20260424172448-0005/0 on worker-202
424171711-172.21.0.3-41825 (172.21.0.3:41825) with 2 core(s)
26/04/24 17:24:48 INFO StandaloneSchedulerBackend: Granted executor ID app-20260424172448-0005/0 on hostPort 172.2
0.3:41825 with 2 core(s), 1024.0 MiB RAM
26/04/24 17:24:48 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferServi
' on port 40039.
26/04/24 17:24:48 INFO NettyBlockTransferService: Server created on a32ce2016d7c:40039
26/04/24 17:24:48 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replica
on policy
26/04/24 17:24:48 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, a32ce2016d7c, 40039, No
)
26/04/24 17:24:48 INFO BlockManagerMasterEndpoint: Registering block manager a32ce2016d7c:40039 with 434.4 MiB RAM
BlockManagerId(driver, a32ce2016d7c, 40039, None)
26/04/24 17:24:48 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, a32ce2016d7c, 40039, Nor
26/04/24 17:24:48 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, a32ce2016d7c, 40039, None)
26/04/24 17:24:48 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20260424172448-0005/0 is now RUNN
G
26/04/24 17:24:48 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reache
minRegisteredResourcesRatio: 0.0
26/04/24 17:24:50 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when
e query didn't fail: /tmp/temporary-47df93c0-85ab-4b59-9365-9c24d997c307. If it's required to delete it under any
rcumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting
mp checkpoint folder is best effort.
26/04/24 17:24:50 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/D
asets and will be disabled.
-------------------------------------------
Batch: 0
-------------------------------------------
+------------------------------------------+-----+
|window                                    |count|
+------------------------------------------+-----+
|{2022-08-20 19:20:00, 2022-08-20 19:21:00}|1980 |
|{2022-08-20 19:51:00, 2022-08-20 19:52:00}|1744 |
|{2022-08-20 15:00:00, 2022-08-20 15:01:00}|1342 |
|{2022-08-20 18:08:00, 2022-08-20 18:09:00}|2004 |
|{2022-08-20 19:38:00, 2022-08-20 19:39:00}|2009 |
|{2022-08-20 15:40:00, 2022-08-20 15:41:00}|2006 |
|{2022-08-20 15:49:00, 2022-08-20 15:50:00}|1850 |
|{2022-08-13 09:10:00, 2022-08-13 09:11:00}|1920 |
|{2022-08-20 15:55:00, 2022-08-20 15:56:00}|2044 |
|{2022-08-13 14:00:00, 2022-08-13 14:01:00}|1337 |
|{2022-08-20 19:57:00, 2022-08-20 19:58:00}|1819 |
|{2022-08-20 15:14:00, 2022-08-20 15:15:00}|1839 |
|{2022-08-20 18:11:00, 2022-08-20 18:12:00}|1836 |
|{2022-08-20 15:25:00, 2022-08-20 15:26:00}|2084 |
|{2022-08-13 14:39:00, 2022-08-13 14:40:00}|2077 |
|{2022-08-13 14:50:00, 2022-08-13 14:51:00}|1947 |
|{2022-08-13 14:19:00, 2022-08-13 14:20:00}|1938 |
|{2022-08-20 19:11:00, 2022-08-20 19:12:00}|1832 |
|{2022-08-20 19:35:00, 2022-08-20 19:36:00}|2092 |
|{2022-08-20 18:19:00, 2022-08-20 18:20:00}|2023 |
+------------------------------------------+-----+
only showing top 20 rows
```



Q10:

Summary: Extend the Tumbling Window application from Q9 by adding a sub-category. The data is now grouped by both the **5-minute Tumbling Window** and the vehicle **Classification** (`CLASSIFICAÇÃO`), providing a breakdown of vehicle types per 5-minute time block. **Python File:** `tumbling_type_count.py`
Command Executed:

```
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master
spark://spark:7077 --total-executor-cores 2 --packages
org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2
/src/streaming/tumbling_type_count.py"
```

```
Terminal - abdallah@abdallah-Laptop: ~
File  Edit  View  Terminal  Tabs  Help
26/04/24 17:33:39 INFO StandaloneAppClient$ClientEndpoint: Connecting to master spark://spark:7077...
26/04/24 17:33:39 INFO TransportClientFactory: Successfully created connection to spark/172.21.0.2:7077 after 22 ms
(0 ms spent in bootstraps)
26/04/24 17:33:39 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20260424173339-0000
26/04/24 17:33:39 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService
' on port 34403.
26/04/24 17:33:39 INFO NettyBlockTransferService: Server created on a32ce2016d7c:34403
26/04/24 17:33:39 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replicati
on policy
26/04/24 17:33:39 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, a32ce2016d7c, 34403, None
)
26/04/24 17:33:39 INFO BlockManagerMasterEndpoint: Registering block manager a32ce2016d7c:34403 with 434.4 MiB RAM,
BlockManagerId(driver, a32ce2016d7c, 34403, None)
26/04/24 17:33:40 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, a32ce2016d7c, 34403, None)
26/04/24 17:33:40 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, a32ce2016d7c, 34403, None)
26/04/24 17:33:40 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20260424173339-0000/0 on worker-20260
424173329-172.21.0.3-39525 (172.21.0.3:39525) with 2 core(s)
26/04/24 17:33:40 INFO StandaloneSchedulerBackend: Granted executor ID app-20260424173339-0000/0 on hostPort 172.21.
0.3:39525 with 2 core(s), 1024.0 MiB RAM
26/04/24 17:33:40 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20260424173339-0000/0 is now RUNNIN
G
26/04/24 17:33:40 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached
minRegisteredResourcesRatio: 0.0
26/04/24 17:33:42 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when th
e query didn't fail: /tmp/temporary-c7487e52-4487-4ca2-8723-40266826abc8. If it's required to delete it under any ci
rcumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting te
mp checkpoint folder is best effort.
26/04/24 17:33:42 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Dat
asets and will be disabled.
-------------------------------------------
Batch: 0
-------------------------------------------
+------------------------------------------+------------------+-----+
|window                                    |CLASSIFICAÇÃO     |count|
+------------------------------------------+------------------+-----+
|{2022-08-13 14:50:00, 2022-08-13 14:55:00}|CAMINHÃO / ÔNIBUS |341  |
|{2022-08-20 18:25:00, 2022-08-20 18:30:00}|MOTO              |1146 |
|{2022-08-13 09:10:00, 2022-08-13 09:15:00}|INDEFINIDO        |3    |
|{2022-08-20 18:35:00, 2022-08-20 18:40:00}|CAMINHÃO / ÔNIBUS |244  |
|{2022-08-20 15:00:00, 2022-08-20 15:05:00}|AUTOMÓVEL         |7866 |
|{2022-08-13 09:15:00, 2022-08-13 09:20:00}|AUTOMÓVEL         |1369 |
|{2022-08-13 09:15:00, 2022-08-13 09:20:00}|MOTO              |199  |
|{2022-08-20 15:15:00, 2022-08-20 15:20:00}|AUTOMÓVEL         |8523 |
|{2022-08-20 18:30:00, 2022-08-20 18:35:00}|CAMINHÃO / ÔNIBUS |278  |
|{2022-08-20 19:55:00, 2022-08-20 20:00:00}|AUTOMÓVEL         |7925 |
|{2022-08-20 18:55:00, 2022-08-20 19:00:00}|INDEFINIDO        |22   |
|{2022-08-20 18:45:00, 2022-08-20 18:50:00}|MOTO              |1198 |
|{2022-08-20 15:00:00, 2022-08-20 15:05:00}|INDEFINIDO        |1    |
|{2022-08-20 15:45:00, 2022-08-20 15:50:00}|AUTOMÓVEL         |8087 |
|{2022-08-13 14:10:00, 2022-08-13 14:15:00}|CAMINHÃO / ÔNIBUS |322  |
|{2022-08-20 18:50:00, 2022-08-20 18:55:00}|INDEFINIDO        |20   |
|{2022-08-20 18:30:00, 2022-08-20 18:35:00}|MOTO              |1114 |
|{2022-08-20 15:50:00, 2022-08-20 15:55:00}|MOTO              |1114 |
|{2022-08-20 15:40:00, 2022-08-20 15:45:00}|CAMINHÃO / ÔNIBUS |362  |
|{2022-08-13 14:30:00, 2022-08-13 14:35:00}|MOTO              |1245 |
+------------------------------------------+------------------+-----+
only showing top 20 rows
```

Q11:

Summary: Implement a **30-minute Sliding Window** that updates (slides forward) every **5 minutes**. This allows records to be counted across multiple overlapping historical windows. The output is configured with `truncate=false` so the full timestamp arrays of the overlapping windows are completely visible in the console. **Python File:** `sliding_count.py` **Command Executed:**

```
docker exec -it spark-master bash -c "/opt/spark/bin/spark-submit --master
spark://spark:7077 --total-executor-cores 2 --packages
```

```
  org.apache.spark:spark-sql-kafka-0-10_2.12:3.3.2
  /src/streaming/sliding_count.py"
```

```
Terminal - abdallah@abdallah-Laptop: ~
File  Edit  View  Terminal  Tabs  Help
26/04/24 17:36:05 INFO StandaloneAppClient$ClientEndpoint: Connecting to master spark://spark:7077...
26/04/24 17:36:05 INFO TransportClientFactory: Successfully created connection to spark/172.21.0.2:7077 after 20 ms
(0 ms spent in bootstraps)
26/04/24 17:36:05 INFO StandaloneSchedulerBackend: Connected to Spark cluster with app ID app-20260424173605-0000
26/04/24 17:36:05 INFO Utils: Successfully started service 'org.apache.spark.network.netty.NettyBlockTransferService
' on port 33537.
26/04/24 17:36:05 INFO NettyBlockTransferService: Server created on a32ce2016d7c:33537
26/04/24 17:36:05 INFO BlockManager: Using org.apache.spark.storage.RandomBlockReplicationPolicy for block replicati
on policy
26/04/24 17:36:05 INFO BlockManagerMaster: Registering BlockManager BlockManagerId(driver, a32ce2016d7c, 33537, None
)
26/04/24 17:36:05 INFO StandaloneAppClient$ClientEndpoint: Executor added: app-20260424173605-0000/0 on worker-20260
424173555-172.21.0.3-45897 (172.21.0.3:45897) with 2 core(s)
26/04/24 17:36:05 INFO StandaloneSchedulerBackend: Granted executor ID app-20260424173605-0000/0 on hostPort 172.21.
0.3:45897 with 2 core(s), 1024.0 MiB RAM
26/04/24 17:36:05 INFO BlockManagerMasterEndpoint: Registering block manager a32ce2016d7c:33537 with 434.4 MiB RAM,
BlockManagerId(driver, a32ce2016d7c, 33537, None)
26/04/24 17:36:05 INFO BlockManagerMaster: Registered BlockManager BlockManagerId(driver, a32ce2016d7c, 33537, None)
26/04/24 17:36:05 INFO BlockManager: Initialized BlockManager: BlockManagerId(driver, a32ce2016d7c, 33537, None)
26/04/24 17:36:05 INFO StandaloneAppClient$ClientEndpoint: Executor updated: app-20260424173605-0000/0 is now RUNNIN
G
26/04/24 17:36:05 INFO StandaloneSchedulerBackend: SchedulerBackend is ready for scheduling beginning after reached
minRegisteredResourcesRatio: 0.0
26/04/24 17:36:08 WARN ResolveWriteToStream: Temporary checkpoint location created which is deleted normally when th
e query didn't fail: /tmp/temporary-bdda3f72-b7ad-4fa2-af85-a83554621f99. If it's required to delete it under any ci
rcumstances, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Important to know deleting te
mp checkpoint folder is best effort.
26/04/24 17:36:08 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Dat
asets and will be disabled.
-------------------------------------------
Batch: 0
-------------------------------------------
+-----------------------------------------+-----+
|window                                   |count|
+-----------------------------------------+-----+
|{2022-08-20 17:40:00, 2022-08-20 18:10:00}|18273|
|{2022-08-13 14:40:00, 2022-08-13 15:10:00}|38604|
|{2022-08-20 14:35:00, 2022-08-20 15:05:00}|9153 |
|{2022-08-13 14:30:00, 2022-08-13 15:00:00}|58459|
|{2022-08-20 19:25:00, 2022-08-20 19:55:00}|58631|
|{2022-08-13 14:05:00, 2022-08-13 14:35:00}|60444|
|{2022-08-20 19:05:00, 2022-08-20 19:35:00}|60839|
|{2022-08-20 17:35:00, 2022-08-20 18:05:00}|8491 |
|{2022-08-20 18:40:00, 2022-08-20 19:10:00}|61435|
|{2022-08-20 18:05:00, 2022-08-20 18:35:00}|59493|
|{2022-08-20 14:50:00, 2022-08-20 15:20:00}|38564|
|{2022-08-20 17:50:00, 2022-08-20 18:20:00}|37855|
|{2022-08-13 13:50:00, 2022-08-13 14:20:00}|38883|
|{2022-08-20 18:50:00, 2022-08-20 19:20:00}|60833|
|{2022-08-20 15:20:00, 2022-08-20 15:50:00}|58079|
|{2022-08-13 14:45:00, 2022-08-13 15:15:00}|28911|
|{2022-08-20 15:10:00, 2022-08-20 15:40:00}|58422|
|{2022-08-13 14:00:00, 2022-08-13 14:30:00}|59316|
|{2022-08-20 18:35:00, 2022-08-20 19:05:00}|61090|
|{2022-08-20 15:15:00, 2022-08-20 15:45:00}|58588|
+-----------------------------------------+-----+
only showing top 20 rows
```